

Object Oriented Programming for ML

Daniele Russica



Agenda and Learning Objectives:

- A gentle introduction
- Object Oriented Programming in action
- Real life examples

Why should you care?

- **PyTorch and scikit-learn** are fundamentally an **OOP** framework.



Why should you care?

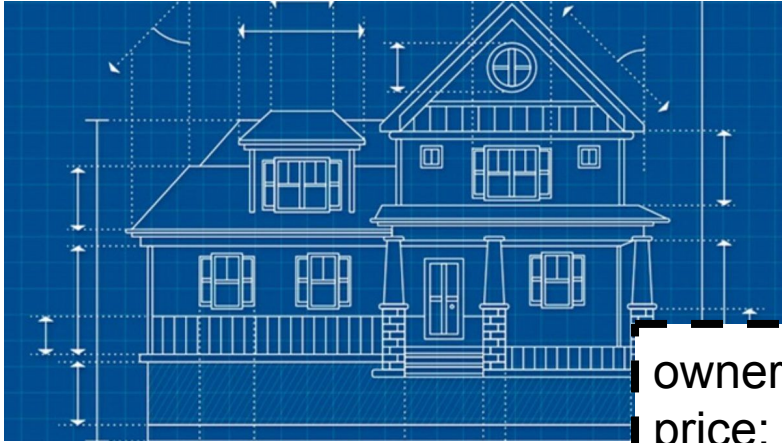
- **PyTorch and scikit-learn** are fundamentally an **OOP framework**.



```
1  model = MyFancyNN()  
2  ...  
3  optimizer = Adam(model.parameters())  
4  ...  
5  dataset = MyDataset(...)  
6  ...  
7  model.train()  
8  ...  
9  model.eval()
```

What even is a class? what about an object?

A class is a blueprint



owner: string
price: int
lights_on: bool

What even is a class? what about an object?

A class is a blueprint



owner: string
price: int
lights_on: bool

An object is an *instance* of a class

owner: **Alice**
price: **700k**
lights_on: **true**



owner: **Bob**
price: **500k**
lights_on: **true**



owner: **Charlie**
price: **600k**
lights_on: **false**

Classes and objects in python

A class is a blueprint

```
1 class House:
2     def __init__(self, owner, cost, lights_on=False):
3         self.owner = owner
4         self.cost = cost
5         self.lights_on = lights_on
6
7     def turn_onLights(self):
8         self.lights_on = True
9
10    def turn_offLights(self):
11        self.lights_on = False
12
```

attributes

methods

What about “self”?!

An object is an *instance* of a class



```
1 house1 = House(owner="Alice",
2                 cost=700_000,
3                 lights_on=True)
```



```
5 house2 = House("Bob", 500_000, True)
```

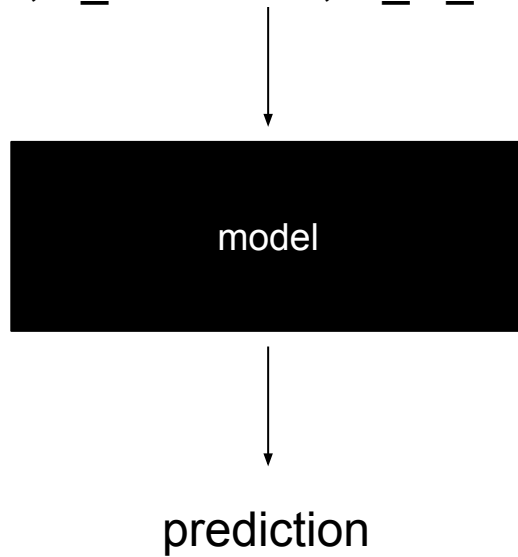
Agenda and Learning Objectives:

- ~~What a **class** and **object** are.~~
- Object Oriented Programming in action
- Real life examples

Our simple running example: a model to predict house prices.

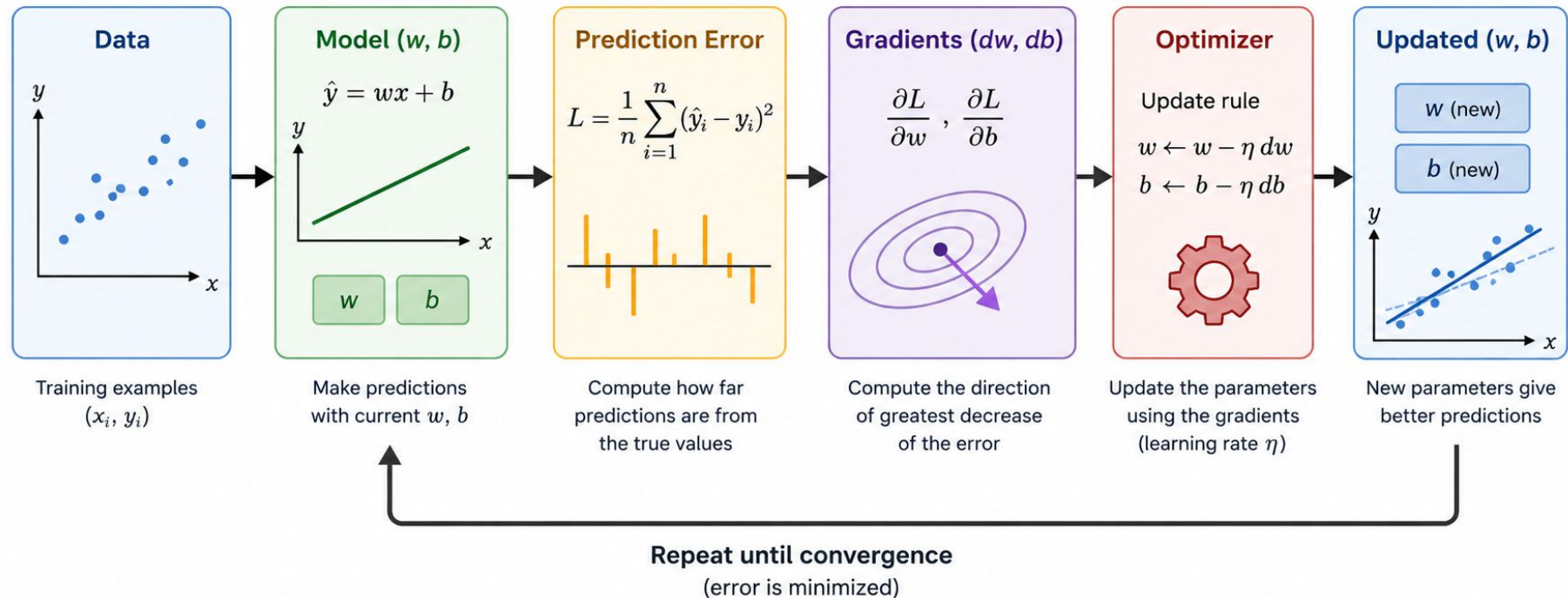
- GOAL: create a model that takes as input house features and predict the price of the house:

(sq_meters, n_bedrooms, is_in_zurich, ...)



Predicting house prices

- The standard training loop acts as follows:



Let's start simple! Fix a line: $y=wx+b$ using plain Gradient Descent

```
1 class LinearRegression:
2     def __init__(self):
3         self.w = 0.0
4         self.b = 0.0
5
6     def predict(self, x):
7         return self.w * x + self.b
8
9     def gradients(self, x, y):
10        y_pred = self.predict(x)
11
12        dw = 2 * np.mean((y_pred - y) * x)
13        db = 2 * np.mean(y_pred - y)
14
15        return [dw, db]
16
17    def parameters(self):
18        return [self.w, self.b]
```

Let's start simple! Fix a line: $y=wx+b$ using plain Gradient Descent

```
1 class LinearRegression:
2     def __init__(self):
3         self.w = 0.0
4         self.b = 0.0
5
6     def predict(self, x):
7         return self.w * x + self.b
8
9     def gradients(self, x, y):
10        y_pred = self.predict(x)
11
12        dw = 2 * np.mean((y_pred - y) * x)
13        db = 2 * np.mean(y_pred - y)
14
15        return [dw, db]
16
17    def parameters(self):
18        return [self.w, self.b]
```

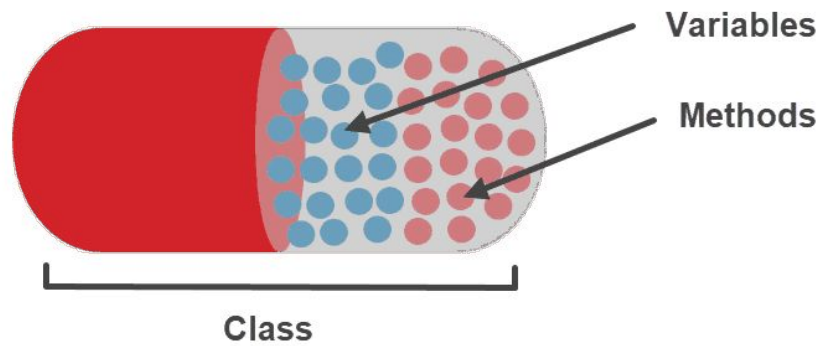
```
1 class GradientDescent:
2     def __init__(self, params, lr=0.01):
3         self.params = params # from our model
4         self.lr = lr
5
6     def step(self, grads):
7         for param, grad in zip(self.params, grads):
8             param -= self.lr * grad
```


Finally we train our model!

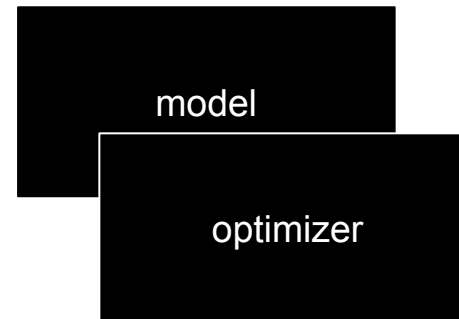
- Instantiate the model and optimizer classes and call methods from it in the training loop:

```
1  x = np.array([1, 2, 3, 4, 5])
2  y = np.array([3, 5, 7, 9, 11])
3
4  model = LinearRegression()
5  optimizer = GradientDescent(model.parameters(), lr=0.05)
6
7  for epoch in range(1000):
8      grads = model.gradients(x, y)
9      optimizer.step(grads)
10
11  # after training model.w and model.b contain the trained parameters
12  # and we can use the model to predict new house prices
13  y_test = model(x_test)
14
```

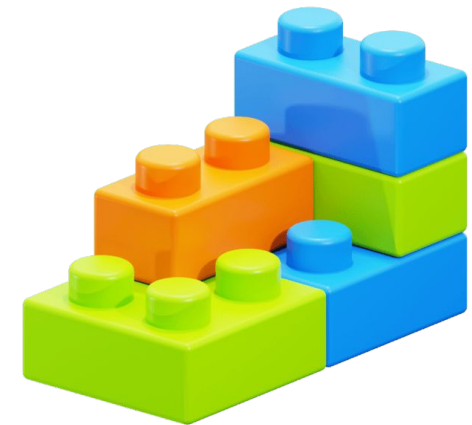
Principles of OOP in action:



Encapsulation &
separation of concerns

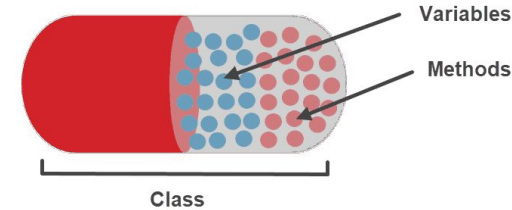


Abstraction



Modularity

Encapsulation and separation of concerns



- Idea: Each class has a single responsibility and contains the state and behavior needed to fulfill it.

GOAL: model the relationship between input and output

```
1 class LinearRegression:
2     def __init__(self):
3         self.w = 0.0
4         self.b = 0.0
5
6     def predict(self, x):
7         return self.w * x + self.b
8
9     def gradients(self, x, y):
10        y_pred = self.predict(x)
11
12        dw = 2 * np.mean((y_pred - y) * x)
13        db = 2 * np.mean(y_pred - y)
14
15        return [dw, db]
16
17    def parameters(self):
18        return [self.w, self.b]
```

state

GOAL: Minimize the prediction error

```
1 class GradientDescent:
2     def __init__(self, params, lr=0.01):
3         self.params = params # from our model
4         self.lr = lr
5
6     def step(self, grads):
7         for param, grad in zip(self.params, grads):
8             param -= self.lr * grad
```

state

behaviour

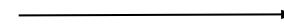
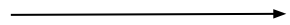
Abstraction

model

optimizer

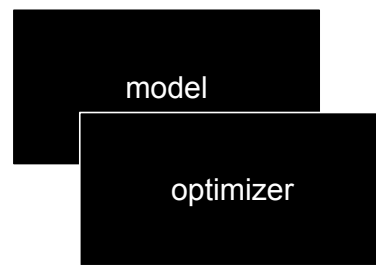
- Idea: OOP allows us to hide away details behind simple interfaces, allowing us users to focus on what the object does instead of how it does it

input: card + pin

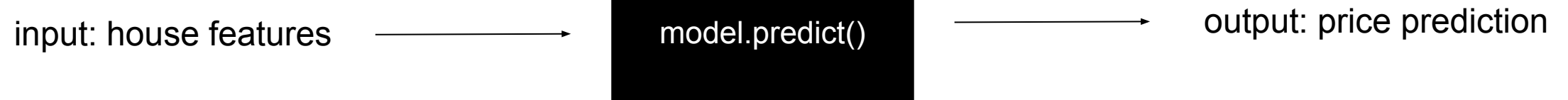


output: cash

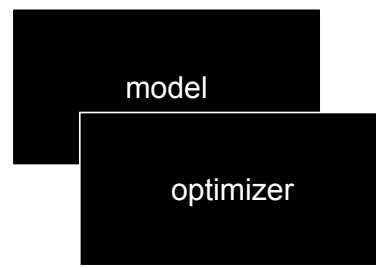
Abstraction



- Idea: OOP allows us to hide away details behind simple interfaces, allowing us users to focus on what the object does instead of how it does it



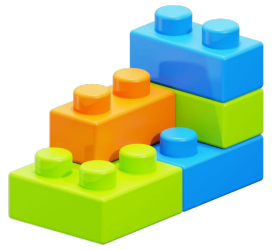
Abstraction



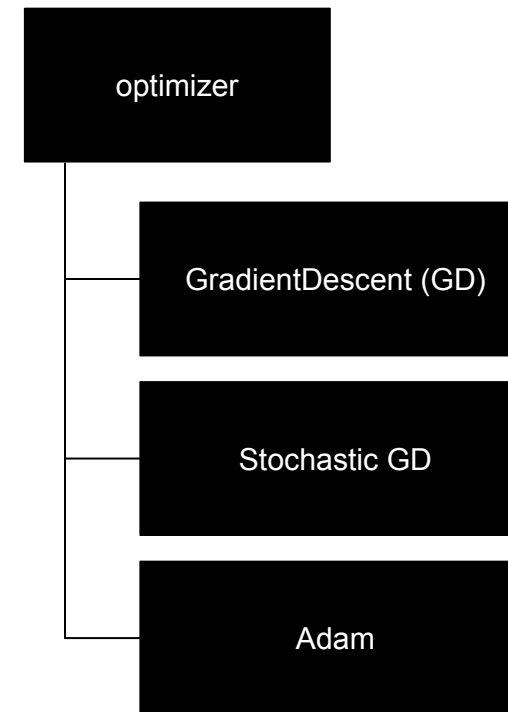
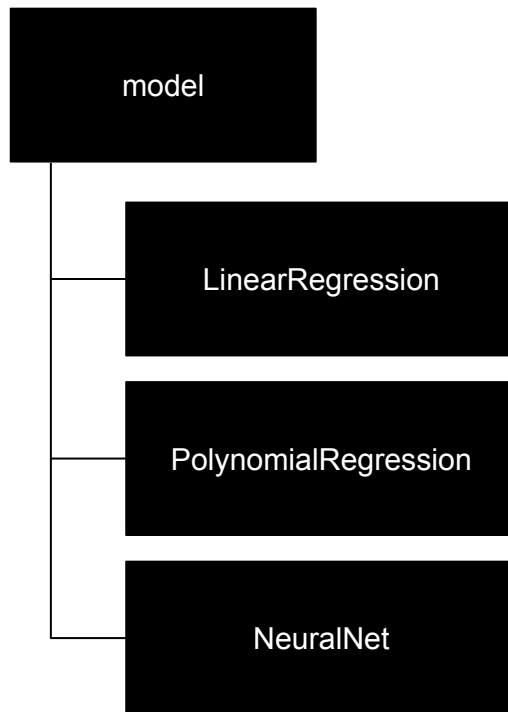
- Idea: OOP allows us to hide away details behind simple interfaces, allowing us users to focus on what the object does instead of how it does it



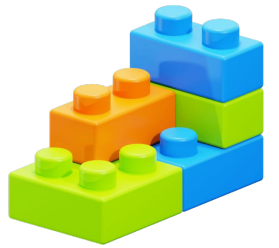
Modularity and Inheritance



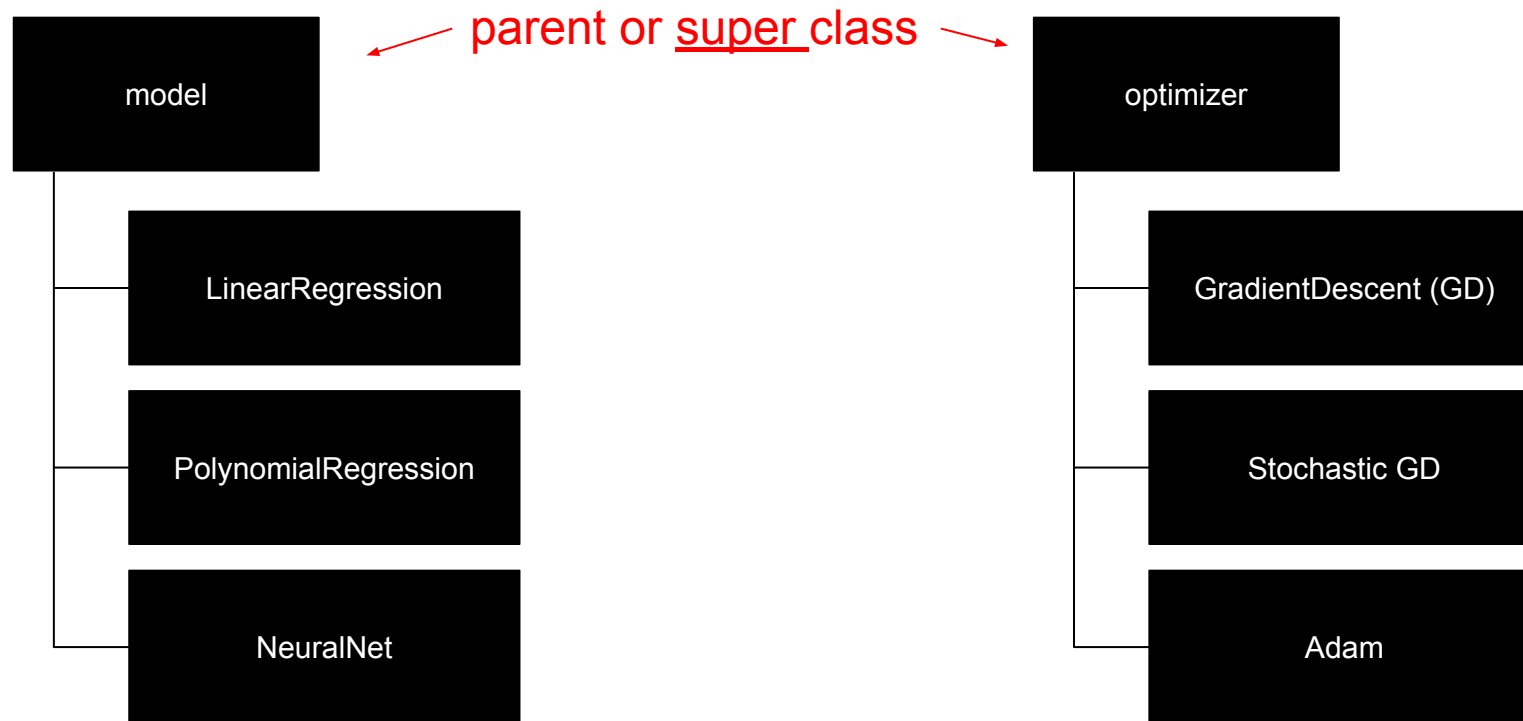
- Idea: by dividing a system into independent components, *each part can be modified or replaced* without affecting the rest of the code.



Modularity and Inheritance



- Idea: by dividing a system into independent components, *each part can be modified or replaced* without affecting the rest of the code.

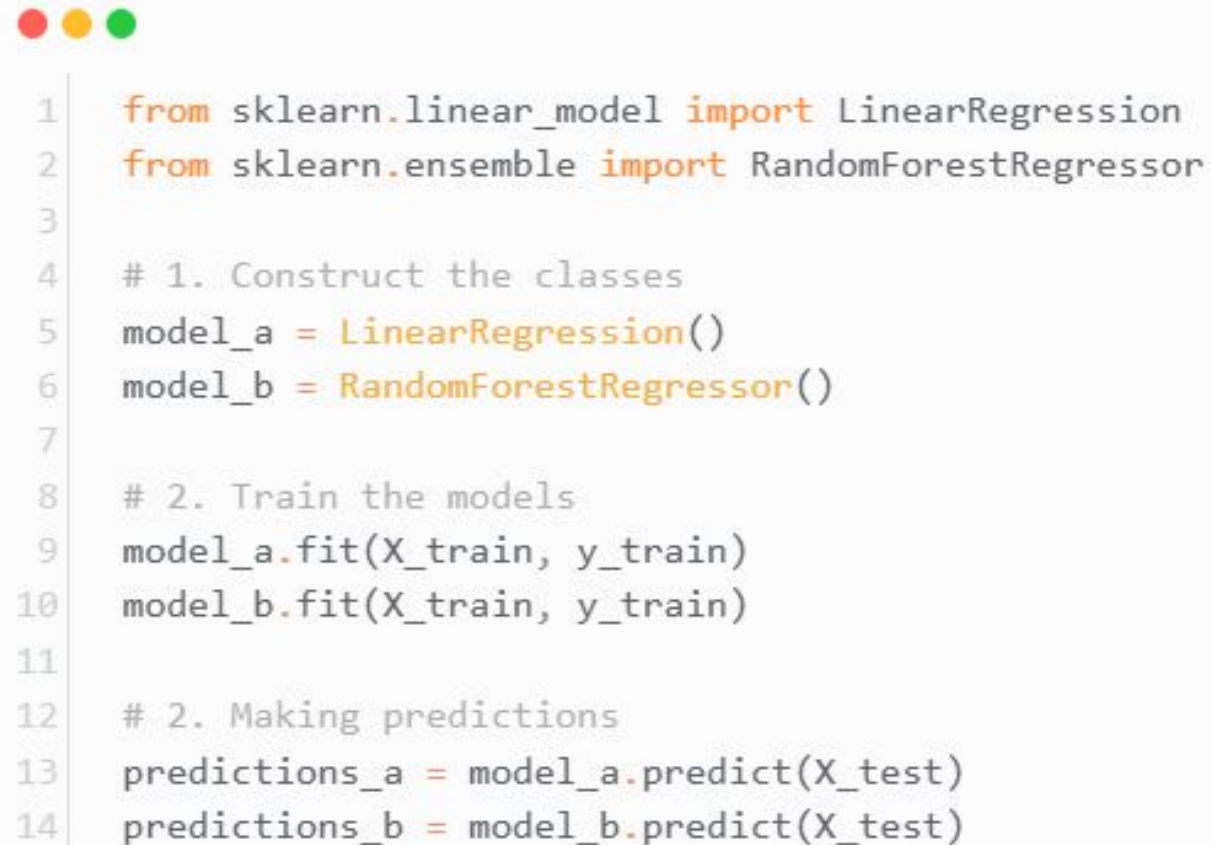


Quiz time!

Agenda and Learning Objectives:

- ~~What a **class** and **object** are.~~
- ~~Object Oriented Programming in action~~
- Real life examples

Real life code snippet examples: Sklearn interface is even simpler!



```
1  from sklearn.linear_model import LinearRegression
2  from sklearn.ensemble import RandomForestRegressor
3
4  # 1. Construct the classes
5  model_a = LinearRegression()
6  model_b = RandomForestRegressor()
7
8  # 2. Train the models
9  model_a.fit(X_train, y_train)
10 model_b.fit(X_train, y_train)
11
12 # 2. Making predictions
13 predictions_a = model_a.predict(X_test)
14 predictions_b = model_b.predict(X_test)
```


Real life code snippet examples: Pytorch

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class HousePricePredictor(nn.Module):
6     def __init__(self):
7         super().__init__()
8         self.linear1 = nn.Linear(in_features=3, out_features=10)
9         self.linear2 = nn.Linear(in_features=10, out_features=1)
10        self.relu = nn.ReLU()
11
12        def forward(self, x):
13            x = self.relu(self.linear1(x))
14            x = self.linear2(x)
15            return x
16
17        model = HousePricePredictor()
18
19        optimizer = optim.Adam(model.parameters(), lr=0.01)
20        criterion = nn.MSELoss()
21
22        optimizer.zero_grad()
23        predictions = model(X_train)
24        loss = criterion(predictions, y_true)
25        loss.backward()
26        optimizer.step()
```

Exercise time!

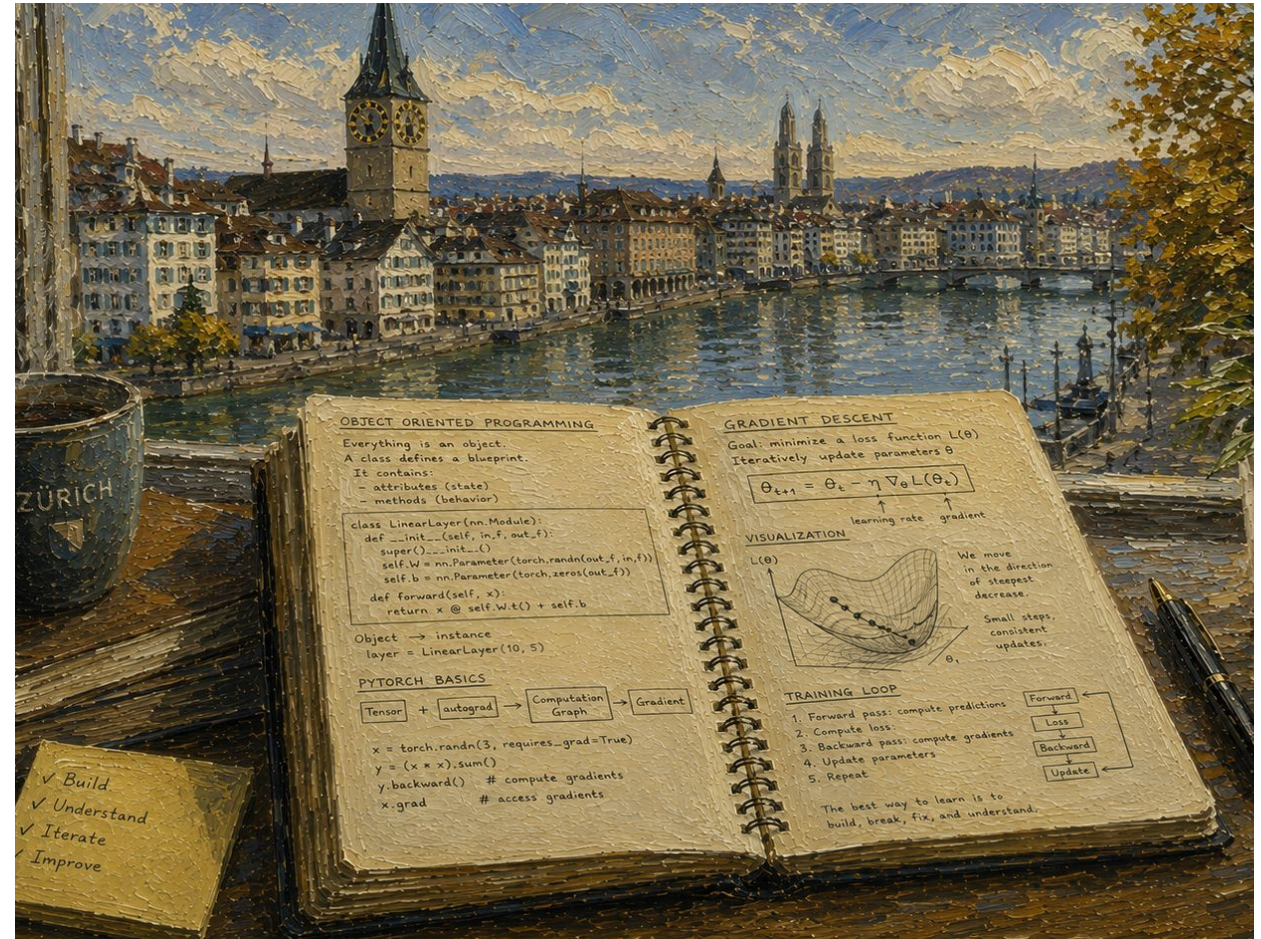
We prepared 2 notebooks for you to practice on OOP:



- Practice theory and your python programming skill and understand OOP better by populating a zoo!
- If you feel proficient enough, feel free to skip this first notebook!

We prepared 2 notebooks for you to practice on OOP:

- Apply OOP to an ML problem
- Build a pytorch-like pipeline yourself
- Understand why modern training loops are built like that



Thank you!

Daniele Russica

